RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Object-Oriented Programming in Java

Mini Project Report

Password Strength Analyzer

By
SAGAR YAJAMAN K N – 1RUA24CSE0391
SRILAKSHMI-1RUA24CSE0475
VEENA-1RUA24CSE0526
VIKAS S- 1RUA24CSE531

Under the guidance of:

Dr. Roopesh R

School of Computer Science and Engineering

RV University, Bangalore

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

CERTIFICATE

Certified that the CS2024 Mini Project work titled "Password Strength Analyzer" Is carried out by *SAGAR YAJAMAN K N(1RUA24CSE0391), SRILAKSHMI(1RUA24CSE0475), VEENA(1RUA24CSE0526*) and *VIKAS S (1RUA24CSE0531)* who is a Bonafide student of the School of Computer Science and Engineering, RV University, Bengaluru, during the year 2025–26. It is certified that all corrections/ suggestions from all the continuous internal evaluations have been incorporated in the project and in this report.

Dr. Roopesh R
Faculty Guide

Dr. K C Narendra
Program Director

Problem statement

With the rapid growth of digital platforms, password security has become a critical concern. Users often create passwords that are easy to remember but highly vulnerable to attacks such as brute force and dictionary attacks. This project aims to develop an **intelligent password strength analyzer** that not only evaluates password strength but also predicts potential vulnerabilities and provides smart recommendations for creating highly secure passwords.

The Project's scope

This project will cover the following:

1.Checking the strength of a password using many different factors
2.Finding weak patterns and passwords that are often used
3.Giving ideas for stronger passwords
4.Showing levels of strength and security feedback

This system can make web apps, banking systems, and login portals safer.

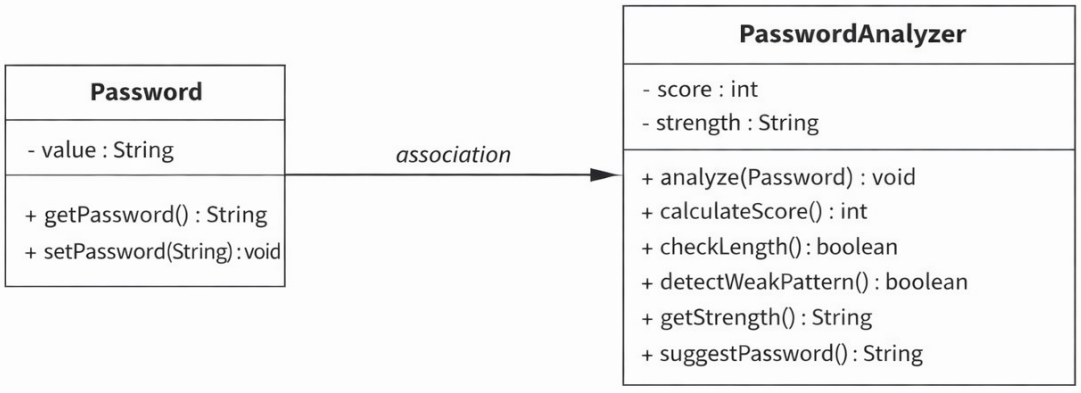
Applications

- Online banking systems
- Social media platforms
- College portals
- E-commerce websites

Advantages

- Improves password security
- Easy to use and implement
- Provides instant feedback
- Helps prevent unauthorized access

Class Diagram



The class diagram above shows how the Password Strength Analyzer system is set up. There are two main classes: Password and PasswordAnalyzer. The Password class keeps track of and stores the user's password. The PasswordAnalyzer class has methods that look at different things, like length, complexity, and pattern detection, to see how strong the password is.

There is an association between the classes, and the analyzer uses the password object to do its work. This design is based on object-oriented principles like encapsulation and modularity, which make the system efficient and easy to keep up with.

Source Code

```
import java.util.*;

class Password {
   private String value;

   public String getPassword() {
      return value;
   }

   public void setPassword(String value) {
      this.value = value;
   }
}

class PasswordAnalyzer {
   private int score;
   private String strength;

   public void analyze(Password p) {
      String password = p.getPassword();
      score = calculateScore(password);

      if (detectWeakPattern(password)) {
         score -= 20;
      }

      strength = getStrength(score);

      System.out.println("Password Score: " + score + "/100");
      System.out.println("Password Strength: " + strength);

      if (score < 70) {
         System.out.println("Suggested Password: " + suggestPassword(password));
      }
   }

   private int calculateScore(String password) {
```

```
        int score = 0;

        if (password.length() >= 8) score += 20;
        if (password.matches(".*[A-Z].*")) score += 20;
        if (password.matches(".*[a-z].*")) score += 20;
        if (password.matches(".*[0-9].*")) score += 20;
        if (password.matches(".*[!@#$%^&*()].*")) score += 20;

        return score;
    }

    private boolean detectWeakPattern(String password) {
        String[] weakPatterns = {"123", "password", "abc", "qwerty"};

        for (String pattern : weakPatterns) {
            if (password.toLowerCase().contains(pattern)) {
                return true;
            }
        }
        return false;
    }

    private String getStrength(int score) {
        if (score < 40)
            return "Weak";
        else if (score < 70)
            return "Medium";
        else if (score < 90)
            return "Strong";
        else
            return "Very Strong";
    }

    private String suggestPassword(String password) {
        return password + "@2026!";
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Password p = new Password();
        PasswordAnalyzer analyzer = new PasswordAnalyzer();

        System.out.print("Enter Password: ");
        p.setPassword(sc.nextLine());

        analyzer.analyze(p);

        sc.close();
    }
}
```

Results :Screen shot

Output

```
Enter Password: abc123
Password Score: 20/100
Password Strength: Weak
Suggested Password: abc123@2026!
```

```
Enter Password: abc@2026
Password Score: 60/100
Password Strength: Medium
Suggested Password: abc@2026@2026!
```

```
Enter Password: Abc@2026
Password Score: 80/100
Password Strength: Strong
```

Conclusion

The Password Strength Analyzer project is a great example of how to use object-oriented programming to make a useful security-based app. The system checks the strength of passwords by looking at things like their length, variety of characters, and ability to find patterns.

It helps people understand why they should make strong passwords and gives them ideas for how to make weak ones better. The system is easy to understand, maintain, and add to because it uses modular design.

This project raises awareness about password security as a whole, and it could be turned into real-time authentication systems for apps like banking, social media, and websites.

Future Enhancements

The system can be further improved by:

- Integrating it with real-time login systems
- Adding a graphical user interface (GUI)
- Using AI techniques for smarter password prediction
- Implementing encryption for enhanced security

Overall Outcome

In conclusion, the project not only strengthens programming skills but also creates awareness about cybersecurity practices. It can be extended into various applications such as banking systems, social media platforms, and secure authentication systems

References

1. Oracle Java Documentation – Regular Expressions
   https://docs.oracle.com/javase/tutorial/essential/regex/

2. GeeksforGeeks – Password Strength Checker using Regex
   https://www.geeksforgeeks.org/check-strength-of-password/

3. TutorialsPoint – Java Regular Expressions
   https://www.tutorialspoint.com/java/java_regular_expressions.htm

4. Oracle Java Documentation – Classes and Objects
   https://docs.oracle.com/javase/tutorial/java/concepts/

5. Stack Overflow – Java Regex and String Matching
   https://stackoverflow.com